

svmodt: An R Package for Linear SVM-Based Oblique Decision Trees

by Aneesh Agarwal, Jack Jewson, and Erik Sverdrup

Abstract Decision trees are widely used for classification tasks due to their interpretability, but traditional axis-aligned splits often require deep trees to approximate complex decision boundaries. We introduce *svmodt*, an R package implementing SVM-based oblique decision trees (SVMODT) that uses linear Support Vector Machines at each node to create multivariate splits. The package supports dynamic feature selection strategies, node-specific class weighting for handling imbalanced data, and feature diversity mechanisms through penalization. We demonstrate that SVMODT outperforms axis-parallel decision trees while also maintaining shallower and more interpretable tree structures. The package includes comprehensive visualization tools, detailed documentation, and is freely available on GitHub.

1 Introduction

Decision trees remain widely used in machine learning because they are interpretable, straightforward to apply, and can accommodate both categorical and numerical predictors with minimal preprocessing. Classical algorithms such as CART (Breiman et al., 1984) and C4.5 (Quinlan, 2014) employ axis-aligned (univariate) splits that partition the feature space along coordinate axes, thereby preserving transparency in the resulting decision rules. Consider a supervised learning task where we wish to predict a categorical or continuous response $y \in \mathcal{Y}$ from d -predictor variables $x \in \mathcal{X} \subseteq \mathbb{R}^d$. In this setting an axis parallel split constitutes selecting one features $j \in \{1, \dots, d\}$ and a splitting value $\theta \in \mathbb{R}$ and splits according to the test $x_j \leq \theta$. Throughout this work, we focus exclusively on the supervised learning problem of predicting y from d -dimensional predictors x , where the goal is to learn a decision function that maps $x \in \mathbb{R}^d \rightarrow y$.

In R, traditional decision trees are readily implemented via packages such as *rpart*, *tree*, and *party*. However, the limited split structure of axis-parallel decision trees can promote overfitting, producing duplicated subtrees and repeated evaluations of the same predictor, which reduces efficiency and weakens generalization (Pagallo and Haussler, 1990). Oblique decision trees mitigate these limitations by permitting splits on linear combinations of features; for example, a single oblique test of the form $\sum_{j=1}^d w_j x_j \leq \theta$, where $w = (w_1, \dots, w_d) \in \mathbb{R}^d$, can often replace multiple axis-aligned tests, producing more compact and geometrically simpler partitions of the feature space. Oblique trees are available in R through packages such as *ODRF*, *aorsf*, and *oblique.tree*. Empirical studies across diverse data sets show that oblique decision trees often achieve higher accuracy and yield more compact models than axis-parallel trees, though with a modest reduction in interpretability. (Cañete et al., 2021).

Figure 1 contrasts an axis-parallel decision tree with an axis-oblique decision tree. The axis-parallel tree displays (left panel) clear overfitting and a high rate of training misclassification, reflecting its inability to capture interactions among predictors that determine class membership. In contrast, the oblique tree (right panel) yields geometrically simpler decision boundaries and substantially fewer misclassifications on the training set, indicating a better fit to the underlying multivariate relationships. Together, these panels illustrate how oblique splits can more effectively represent linear combinations of features and thereby reduce spurious complexity and error in tree-based classifiers.

Support Vector Machines (SVMs), introduced by Cortes and Vapnik (1995), provide a principled way to obtain oblique splits by finding margin-maximizing hyperplanes. This margin-based approach enhances the generalization ability of the model, making SVMs robust and effective for many real-world problems (Cristianini, 2000). The idea of fitting SVMs at tree nodes was formalized by Bennett and Blue (1998) and extended to multiclass settings and other refinements in subsequent work (Takahashi and Abe, 2002; Bala and Agrawal, 2011; Ganaie et al., 2022). Python's *STree* implements SVM-node oblique trees using *scikit-learn*'s *SVC* (Montañana et al., 2021; Pedregosa et al., 2011), while Java tools such as *Weka* offer components that can be adapted for hierarchical SVM trees (Menkovski et al., 2008); however, fully featured, native R implementations remain limited.

To address this gap, we advance two primary contributions. First, we introduce a fully native R implementation of an *STree*-style oblique decision tree that operates independently of Python. This implementation provides a production-ready API and leverages vectorized computation to ensure efficiency and seamless integration within the R ecosystem. Second, we develop *SVM-ODT*, an enhanced SVM-based oblique decision tree that extends the original *STree* framework through several practical innovations. These include feature sub-setting, embedded feature selection, feature penalization, and node-specific class weighting. Collectively, these enhancements are designed to

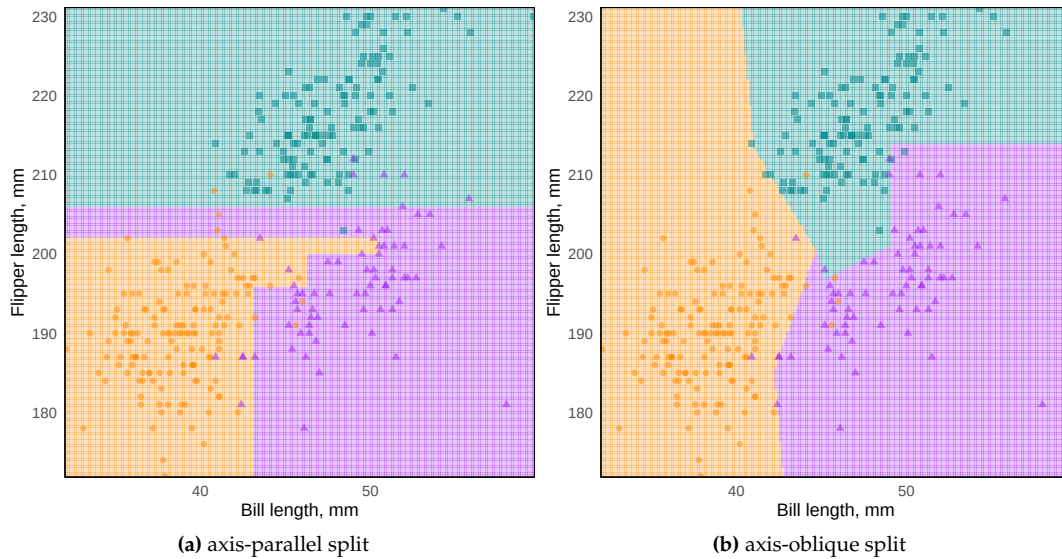


Figure 1: Comparison of Axis-parallel and Axis-oblique splits on Palmerpenguins dataset.

improve predictive performance, interpretability, and robustness across both binary and multiclass classification settings.

Together, these contributions aim to make the representational strengths of SVM-driven oblique splits accessible to R programmers while simultaneously addressing key computational and methodological limitations present in existing approaches.

2 Background

Decision Trees

Decision Trees (DTs) are interpretable classification models that represent their decision-making process through a hierarchical, tree-like structure. This structure comprises of internal nodes containing splitting criteria that divide up the predictor space and terminal (leaf) nodes corresponding to predicted outcomes (class labels and probabilities). The nodes are connected by directed edges, each representing a possible True or False outcome of a splitting criterion for observations whose feature reach this node. Formally, a DT can be expressed as a rooted, directed tree $T = (G(V, E), v_1)$, where V denotes the set of nodes, E represents the set of edges linking these nodes, and v_1 is the root node.

Decision tree algorithms can be categorized based on whether the same type of test is applied at all internal nodes. **Homogeneous trees** employ a single algorithm throughout (e.g., univariate or multivariate splits), whereas **hybrid trees** allow different algorithms such as linear discriminant functions, k -nearest neighbors, or univariate splits that can be used in different sub-trees (Brodley and Utgoff, 1995). Hybrid trees exploit the principle of *selective superiority*, allowing subsets of the data to be modeled by the most appropriate classifier, thereby improving flexibility and accuracy. However, hybrid tree models typically incur substantial computational overhead due to their structural complexity, and therefore, in this paper we restrict our focus to homogeneous trees.

Axis-Parallel Decision Trees

Axis-Parallel Decision trees represent hyper-planes dividing the instance space into several disjoint axis-parallel regions. Axis-parallel decision trees, such as CART and C4.5, represent two of the most widely used algorithms for classification tasks. The **CART (Classification and Regression Trees)** algorithm employs a binary recursive partitioning procedure at capable of handling both continuous and categorical variables as predictors or targets. At each node, the algorithm systematically evaluates every available variable $j \in \{1, \dots, d\}$ and all of its potential split points θ i.e. between any two observations, to determine the optimal partition. In R, traditional decision trees are readily implemented via packages such as `rpart`, `tree`, and `party`.

In contrast, **C4.5**, an extension of the earlier **ID3** algorithm (Quinlan, 1986), utilizing information theory measures such as **information gain** and **gain ratio** to select the most informative attribute

for each split (Quinlan, 2014). C4.5 also includes mechanisms to handle missing attribute values by weighting instances according to the proportion of known data and employs an **error-based pruning** to eliminate subtrees that fail to improve predictive performance beyond a threshold (α), thereby mitigating overfitting. Although these techniques are effective across diverse data sets, studies have shown that the choice of pruning strategy and stopping criteria can significantly affect model performance across different domains (Mingers, 1989; Schaffer, 1992).

Axis-parallel decision trees offer strong interpretability but face inherent representational limits. Their constrained split structure often induces overfitting, leading to duplicated subtrees and repeated tests of the same predictor, ultimately reducing efficiency and weakening generalization (Pagallo and Haussler, 1990).

Axis-Oblique Decision Trees

Axis-oblique decision trees extends axis-parallel decision trees by allowing each internal node to perform splits based on linear or nonlinear combinations (Chabbouh et al., 2025) of multiple features. This flexibility enables the tree to form oblique decision boundaries that more accurately partition the instance space. For example, a single multivariate test such as $x + y < 8$ can replace multiple univariate splits needed to approximate the same boundary. Oblique trees are available in R through packages such as **ODRF**, **aorsf**, and **oblique.tree**.

The construction of Axis-oblique decision trees introduces several design considerations, including how to represent multivariate tests, determine their coefficients, select features to include, handle symbolic and missing data, and prune to avoid over-fitting (Brodley and Utgoff, 1995). Various optimization algorithms such as recursive least squares (Young, 1984), the pocket algorithm (Gallant, 1986), or thermal training (Frean, 1990) may be used to estimate the weights. However, Axis-oblique decision trees trade interpretability for representational power and often require additional mechanisms for **local feature selection**, such as *sequential forward selection* (SFS) or *sequential backward elimination* (SBE) (Kittler, 1986).

Although oblique splits introduce additional complexity and reduce interpretability, oblique decision trees retain key advantages of standard decision trees such as sequential split criteria evaluation and transparent decision procedures while offering improved modeling flexibility for complex data sets (Koziol and Wozniak, 2009; Friedl and Brodley, 1997; Liu and Setiono, 1998).

Alternate Approaches to Oblique Decision Boundaries

Employing linear classifiers at internal nodes can yield decision trees that are both accurate and diverse. Because each node receives a different subset of the training data, the most suitable separating hyperplane may vary from node to node. For example, (Menze et al., 2011; Lemmond et al., 2010) use LDA to construct linear splits, though their approach is limited to binary classification. Zhang and Suganthan (2015) adopts MPSVM to generate node-wise linear separators, while Truong (2009) fits multiple logistic-regression-based partitions (specifically $2^{K-1} - 1$ for K classes) and selects the split that best optimizes the impurity criterion.

Projection pursuit methods (Friedman and Tukey, 2006; Kruskal, 1969) have also been used to identify low-dimensional projections that expose structure in high-dimensional data. The **PPtree** algorithm extends this idea into a recursive partitioning framework by optimizing a class-sensitive projection index at each node and then applying standard split rules to the resulting one-dimensional projection. Similarly, Bulò and Kotschieder (2014) introduce a randomized multi-layer perceptron as a node-level splitting function. However, determining an appropriate network complexity remains challenging, as overly flexible models risk substantial overfitting.

Support Vector Machines (SVMs)

Support Vector Machines (SVMs) are powerful supervised learning models used for classification and regression tasks. They aim to determine an optimal separating hyperplane that maximizes the margin between binary classes in the data. This margin-based approach enhances the generalization ability of the model, making SVMs robust and effective for many real-world problems (Cristianini, 2000).

A simplest **linear SVMs** construct a separating hyperplane in an d -dimensional space such that the margin between the classes is maximized. Given a training dataset $\{(x_i, y_i)\}_{i=1}^N$, where $x_i \in \mathbb{R}^p$ and $y_i \in \{-1, +1\}$, the decision function is defined as $f(x) = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$. The optimal hyperplane is the one that maximizes the distance between the closest points of each class (the **support vectors**) and the hyperplane itself (Cortes and Vapnik, 1995). Training an SVM entails solving a convex quadratic programming (QP) problem defined over an $(n \times n)$ kernel matrix, where (n) is the number of training

samples. Owing to the convexity of the objective, the QP admits a unique global optimum and can be solved reliably using standard optimization techniques. While linear classifiers provide useful insights, they are often inadequate for real-world data sets, where classes are not linearly separable. In such cases, SVMs can be extended to create **nonlinear decision boundaries** by mapping the input vectors into a higher-dimensional **feature space** using a nonlinear transformation $\phi: \mathbb{R}^n \rightarrow \mathcal{F}$. The linear transformation is then achieved in the transformed space using $f(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \phi(\mathbf{x}) + b)$.

Although SVMs exhibit strong theoretical foundations and robust generalization capabilities, they present several practical limitations, most notably their lack of interpretability. Model performance is highly dependent on the appropriate selection of hyperparameters such as the regularization term (C) and kernel parameters (e.g., γ), which govern the trade-off between margin maximization and misclassification tolerance (Nanda et al., 2018). For large-scale data sets, training an SVM can become computationally expensive, with both runtime and memory consumption growing quadratically in the number of training samples (Dong et al., 2005). Moreover, SVMs are inherently designed for binary classification, necessitating decomposition strategies such as One-vs-One and One-vs-All for multi-class problems (Hsu and Lin, 2002). Their performance also tends to degrade in imbalanced data settings, where the decision boundary becomes biased toward the majority class (Cervantes et al., 2020).

Oblique Decision Trees with Support Vector Machines

Support Vector Machine-based decision trees (DTSVMs) were first formalized by Bennett and Blue (1998), who adapted Statistical Learning Theory and Structural Risk Minimization to construct binary decision trees in which each internal node is an SVM that partitions the data by an optimal hyperplane. This formulation enabled multivariate, margin-based decisions at every split. Takahashi and Abe (2002) extended the idea to multiclass settings by using a recursive partitioning strategy: the root node isolates the most separable class or classes from the remainder, and the procedure recurses until each leaf contains a single class, thereby covering the entire feature space. Subsequent studies have built upon this foundation to address scalability, optimization, and generalization issues. Optimal Decision Tree SVM (ODT-SVM) (Bala and Agrawal, 2011) introduced split-selection criteria based on Gini index, information gain, and scatter matrix separability to balance tree interpretability and margin-based precision.

STree

More recent approaches embed multiple SVM classifiers directly within each split to handle multi-class problems more efficiently. STree (Montañana et al., 2021) is an oblique multi-class decision tree algorithm that integrates support vector machines (SVMs) to construct single margin-based splits capable of handling multiclass problems. Unlike traditional multi-class tree methods that rely on clustering or ensembles of binary models, STree builds a single decision tree. Its central innovation lies in using SVM-derived hyperplanes at each internal node, enabling more expressive splits than axis-aligned trees while also being interpretable and computationally simple.

Methodology The algorithm generates a binary tree recursively (see Algorithm 5 in Appendix) . At each node:

- 1) Stopping conditions are evaluated to check the depth of the tree and the class purity. If the stopping conditions are met, a leaf node is created, labelled with the most frequent class in the node.
- 2) If the stopping conditions haven't been met the algorithm then selects the best hyperplane to split the data into two partitions: T^+ (positive side) and T^- (negative side).

When all instances at a node belong to more than two classes ($k' > 2$), the approach generates multiple candidate binary splits: For each class label y_i , it constructs a one-vs-rest binary problem: Class y_i vs. all other classes. It trains an SVM for each of these k' problems, resulting in k' hyperplanes H_i . Each hyperplane partitions the data into T_i^+ and T_i^- . The algorithm computes the impurity (using Shannon entropy) of the class distribution within each partition. When only two classes remain, STree trains a single SVM to obtain the maximum-margin hyperplane. Instances are then routed left or right based on the sign of their distance to this hyperplane. STree's performance depends heavily on hyperparameter tuning, including kernel choice (linear, polynomial or Gaussian), gamma, polynomial degree, regularization parameter C , and max iteration.

Synthesis and implications for future work

Across these developments, two recurring themes emerge: (1) leveraging multivariate, oblique decision boundaries to capture complex class geometry, and (2) combining transformation or ensemble strategies (rotations, multiple SVMs) to improve diversity and robustness. These trends point toward hybrid architectures that balance interpretability, margin-based generalization, and computational tractability—an agenda that motivates continued exploration of regularization schemes, split-selection heuristics, and efficient multi-class integration in SVM-based tree models.

3 Contributed Methods

STreeR

Algorithm 1: STreeR: Multi-class OVR SVM-based oblique decision tree

Input: $\mathcal{D}' = \{(\vec{x}_i, y_i)\}_{i=1}^t$: Data
Output: *tree*: the root node of an SVM-based oblique decision tree

```

1 function STree( $\mathcal{D}'$ ):
2   if stopping_condition( $\{y_i\}_{i=1}^t$ ) then
3     return create_leaf_node(mode( $\{y_i\}_{i=1}^t$ )) // Leaf node
4    $k' \leftarrow \text{num\_different\_labels}(\{y_i\}_{i=1}^t)$ 
5    $\mathcal{Y}' \leftarrow \{y'_1, \dots, y'_{k'}\}$  // set of labels
6    $I_{all} \leftarrow I(\{y_i\}_{i=1}^t)$  //  $I(\cdot)$  is an information theory measure
7    $n_{models} \leftarrow k'$  // OVR
8   for  $j = 1$  to  $n_{models}$  do
9     if ( $j = k' = 2$ ) then
10      break // Two labels, one SVM is enough
11      $models[j] \leftarrow \text{SVM}(\mathcal{D}')$  using binary class  $y'_j$  (+) vs rest (-) // OVR
12    $\mathcal{D}^+, \mathcal{D}^- \leftarrow models[j](\vec{x}_j) \quad \forall (\vec{x}) \in \mathcal{D}'$ 
13    $I_j \leftarrow \frac{|\mathcal{D}^+|}{|\mathcal{D}'|} I(\{y_i\}_{i=1}^{|\mathcal{D}^+|}) + \frac{|\mathcal{D}^-|}{|\mathcal{D}'|} I(\{y_i\}_{i=1}^{|\mathcal{D}^-|})$ 
14    $IG_j \leftarrow I_{all} - I_j$ 
15    $b^* = \arg \max_{j=1, \dots, n_{models}} IG_j$ 
16   if  $IG_{b^*} > 0$  then
17     node  $\leftarrow \text{create\_node}(models[b^*])$ 
18     node.left  $\leftarrow \text{STree}(\mathcal{D}^+)$ 
19     node.right  $\leftarrow \text{STree}(\mathcal{D}^-)$ 
20   else
21     return create_leaf_node(mode( $\{y_i\}_{i=1}^t$ )) // No split gain
22   return node

```

A key contribution of this work is the first native R implementation of the Stree algorithm. Because no such implementation currently exists, we reconstructed the method in R using [e1071](#), which interfaces with the LIBSVM C++ backend. This reconstruction provided clarity on the algorithm's internal workflow and ensured fair benchmarking against competing approaches. Algorithm 1 provides a concise summary of our R-based re-implementation of the Stree algorithm.

To verify the accuracy of our R implementation of STree, we conducted a benchmarking comparison against the original Python version using a 10×5 -fold cross-validation scheme applied to 9 benchmark datasets from the UCI Machine Learning Repository. For comparability, both algorithms were run with the same default hyperparameters: cost-complexity $C = 1$, a linear kernel, a maximum of 1×10^7 training iterations, and a maximum depth of 10. Prediction accuracies were averaged across all folds for each dataset.

Across most datasets, STreeR exhibits performance that is effectively identical to its Python counterpart (see Table ??). Although both implementations rely on the same underlying LIBSVM C++ library, Python through the SVC classifier and R through the `e1071::svm` interface, there exist subtle yet meaningful differences in how each framework invokes the underlying routines responsible for fitting the support vector machine decision boundary and applying scaling transformations during

Table 1: Comparison of mean prediction accuracy and median training time for StreeR and STree. Here, N denotes the number of observations, X the number of features, and L the number of classes.

Dataset	N	X	L	StreeR	STree	StreeR(Med)	STree(Med)
WDBC Diagnosis	569	30	2	0.970 $\pm$ 0.004	0.970 $\pm$ 0.003	0.043ms	0.019ms
Iris	150	4	3	0.966 $\pm$ 0.006	0.965 $\pm$ 0.010	0.011ms	0.007ms
Echocardiogram	131	10	2	0.846 $\pm$ 0.004	0.845 $\pm$ 0.008	0.007ms	0.006ms
Fertility	100	9	2	0.876 $\pm$ 0.011	0.874 $\pm$ 0.010	0.003ms	0.005ms
Wine	178	13	3	0.978 $\pm$ 0.008	0.959 $\pm$ 0.008	0.023ms	0.007ms
Cardiotography-3	2126	21	3	0.901 $\pm$ 0.003	0.903 $\pm$ 0.003	0.97ms	0.242ms
Cardiotography-10	2126	21	10	0.792 $\pm$ 0.004	0.795 $\pm$ 0.018	4.391ms	0.474ms
Ionosphere	351	33	2	0.896 $\pm$ 0.009	0.892 $\pm$ 0.015	0.049ms	0.021ms
Dermatology	366	34	6	0.972 $\pm$ 0.006	0.959 $\pm$ 0.004	0.126ms	0.019ms

prediction. These implementation-level discrepancies are likely to contribute to the observed variation in performance.

To mitigate differences attributable to feature scaling, each dataset has been pre-processed in R using an implementation that replicates the behaviour of Python's standard scaling procedure.

Given computational and scope constraints, our re-implementation focuses only on the components of the STree algorithm that are directly relevant to our research objectives. In particular, we restrict our analysis to the one-vs-rest splitting strategy. The alternative one-vs-one scheme is not included, as it scales poorly with the number of classes: for problems with large L , the number of pairwise SVMs grows quadratically, substantially increasing training time without offering clear benefits for the aspects of the algorithm we aim to study. We also implement only a linear kernel, both to maintain the interpretability of the resulting tree structure and to avoid the considerable computational overhead associated with non-linear kernels. At present, the implementation does not include hyperparameters for feature subsetting, although users can replicate this functionality by manually providing a reduced feature set.

Support Vector Machine based Oblique Decision Trees: SVMODT

Algorithm 2: SVMODT: SVM-based Oblique Decision Tree with Enhanced Splitting

Input: $\mathcal{D} = \{(\vec{x}_i, y_i)\}_{i=1}^n$: training dataset, d : current depth, $d_{\max}$: maximum depth, $n_{\min}$: minimum samples, m_{base} : base number of features, $\mathcal{C}$: set of all possible class labels

Output: *tree*: root node of the SVM-based oblique decision tree

```

1 function SVMODT( $\mathcal{D}, d, d_{\max}, n_{\min}, m_{\text{base}}, \text{method}_{\mathcal{F}}, \text{measure}_I, \text{strat}_m, \text{strat}_w, \mathcal{F}_{\text{used}}, \lambda_{\text{pen}}, \tau_I, \mathcal{C}$ ):
2    $n \leftarrow |\mathcal{D}|$ 
3    $\mathcal{Y} \leftarrow \{y_i\}_{i=1}^n$  // Extract labels
4   // Stopping conditions
5   if  $d > d_{\max}$  or  $|\text{unique}(\mathcal{Y})| = 1$  or  $n < n_{\min}$  then
6     return create_leaf_node(mode( $\mathcal{Y}$ ),  $n, \mathcal{C}$ )
7   // Dynamic feature selection and scaling
8    $m_d \leftarrow \text{calculate\_dynamic\_max\_features}(d, m_{\text{base}}, \text{strat}_m)$ 
9    $\mathcal{F}_d \leftarrow \text{select\_features\_with\_penalty}(\mathcal{D}, m_d, \text{method}_{\mathcal{F}}, \mathcal{F}_{\text{used}}, \lambda_{\text{pen}})$ 
10   $\mathbf{X}_{\text{scaled}}, \text{scaler} \leftarrow \text{scale\_node}(\mathcal{D}[\mathcal{F}_d])$ 
11   $\mathcal{F}_d \leftarrow \text{features}(\mathbf{X}_{\text{scaled}})$  // Update to non-constant features
12  if  $|\mathcal{F}_d| = 0$  then
13    return create_leaf_node(mode( $\mathcal{Y}$ ),  $n, \mathcal{C}$ ) // No valid features
14   $k \leftarrow |\text{unique}(\mathcal{Y})|$  // Number of classes at node
15   $I_{\text{parent}} \leftarrow I(\mathcal{Y}, \text{measure}_I)$  // Parent impurity
16  // Binary vs Multiclass handling
17  if  $k = 2$  then
18    ( $\text{best\_model}, \mathcal{I}_L, \mathcal{I}_R, c_{\text{split}}$ )  $\leftarrow \text{BinarySplit}(\mathbf{X}_{\text{scaled}}, \mathcal{Y}, \text{strat}_w)$  // Refer to Algorithm
19    3
20  else
21    ( $\text{best\_model}, \mathcal{I}_L, \mathcal{I}_R, c_{\text{split}}$ )  $\leftarrow$ 
22    MulticlassSplit( $\mathbf{X}_{\text{scaled}}, \mathcal{Y}, \text{strat}_w, I_{\text{parent}}, \text{measure}_I, \tau_I, n$ ) // Refer to Algorithm
23    4
24  if  $\text{best\_model} = \text{null}$  or  $|\mathcal{I}_L| = 0$  or  $|\mathcal{I}_R| = 0$  then
25    return create_leaf_node(mode( $\mathcal{Y}$ ),  $n, \mathcal{C}$ )
26  // Handle small children
27  if  $|\mathcal{I}_L| < n_{\min}$  and  $|\mathcal{I}_R| < n_{\min}$  then
28    return create_leaf_node(mode( $\mathcal{Y}$ ),  $n, \mathcal{C}$ ) // Both children too small
29  // Update used features for penalty mechanism
30   $\mathcal{F}'_{\text{used}} \leftarrow \mathcal{F}_{\text{used}} \cup \mathcal{F}_d$ 
31  // Create internal node
32   $\text{node} \leftarrow \text{create\_internal\_node}(\text{best\_model}, \mathcal{F}_d, \text{scaler}, c_{\text{split}}, d, n)$ 
33  // Recursively build children
34  if  $|\mathcal{I}_L| \geq n_{\min}$  then
35     $\text{node.left} \leftarrow \text{SVMODT}(\mathcal{D}[\mathcal{I}_L], d + 1, d_{\max}, n_{\min}, m_{\text{base}}, \dots, \mathcal{F}'_{\text{used}}, \dots, \mathcal{C})$ 
36  else
37     $\text{node.left} \leftarrow \text{create\_leaf\_node}(\text{mode}(\mathcal{Y}[\mathcal{I}_L]), |\mathcal{I}_L|, \mathcal{C})$ 
38  if  $|\mathcal{I}_R| \geq n_{\min}$  then
39     $\text{node.right} \leftarrow \text{SVMODT}(\mathcal{D}[\mathcal{I}_R], d + 1, d_{\max}, n_{\min}, m_{\text{base}}, \dots, \mathcal{F}'_{\text{used}}, \dots, \mathcal{C})$ 
40  else
41     $\text{node.right} \leftarrow \text{create\_leaf\_node}(\text{mode}(\mathcal{Y}[\mathcal{I}_R]), |\mathcal{I}_R|, \mathcal{C})$ 
42  return node

```

The Support Vector Machine based Oblique Decision Tree (SVMODT) constructs a binary classification decision tree where each internal node v at depth d . The algorithm (see Algorithm 2) is implemented in R using the [e1071](#) performs the following operations:

Key Features

Binary & Multiclass SVM Training At each internal node of the tree, a Support Vector Machine (SVM) is trained to construct an oblique decision boundary that partitions the data into two child nodes. The training procedure differs depending on whether the node contains samples from two classes or more than two classes.

Binary Classification When the node contains exactly two classes ($k = 2$), a binary SVM classifier is trained (see 3) using the selected feature subset $\mathcal{F}_d$. Let $\mathbf{X}_{\text{scaled}}$ denote the standardized feature matrix at the node and $\mathcal{Y}$ the corresponding class labels.

Algorithm 3: Binary Classification Split

Input: $\mathbf{X}_{\text{scaled}}$: scaled features, $\mathcal{Y}$: labels, strat_w : class weighting strategy

Output: (model, $\mathcal{I}_L$, $\mathcal{I}_R$, c_{split}): SVM model, left indices, right indices, split class

```

1 function BinarySplit( $\mathbf{X}_{\text{scaled}}$ ,  $\mathcal{Y}$ ,  $\text{strat}_w$ ):
2    $w_c \leftarrow \text{calculate\_class\_weights}(\mathcal{Y}, \text{strat}_w)$ 
3   model  $\leftarrow \text{fit\_linear\_SVM}(\mathbf{X}_{\text{scaled}}, \mathcal{Y}, w_c)$ 
4   if model = null then
5     return (null,  $\emptyset$ ,  $\emptyset$ , null)
6   f  $\leftarrow \text{model.decision\_values}(\mathbf{X}_{\text{scaled}})$ 
7    $\mathcal{I}_L \leftarrow \{i : f_i > 0\}$ ,  $\mathcal{I}_R \leftarrow \{i : f_i \leq 0\}$ 
8   return (model,  $\mathcal{I}_L$ ,  $\mathcal{I}_R$ , null)

```

The SVM learns a separating hyperplane of the form

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b,$$

where $\mathbf{w}$ represents the weight vector and b the bias term. The hyperplane partitions the data into two subsets according to the value of the decision function. Samples satisfying $f(\mathbf{x}) \leq c_{\text{split}}$ are assigned to the left child node, while those with $f(\mathbf{x}) > c_{\text{split}}$ are assigned to the right child node.

Class weights may optionally be applied during SVM training to address class imbalance. The resulting partition indices $\mathcal{I}_L$ and $\mathcal{I}_R$ are then used to recursively construct the left and right subtrees.

Multiclass Classification When more than two classes are present at the node ($k > 2$), a multiclass SVM strategy is employed (see 4) to generate a binary split. The algorithm evaluates candidate partitions of the class set in order to determine a split that maximizes impurity reduction.

Algorithm 4: Multiclass One-vs-Rest Split

Input: $\mathbf{X}_{\text{scaled}}$: scaled features, $\mathcal{Y}$: labels, strat_w : class weighting strategy, I_{parent} : parent impurity, measure_I : impurity measure, τ_I : minimum information gain, n : number of samples

Output: $(\text{best_model}, \mathcal{I}_L, \mathcal{I}_R, c_{\text{split}})$: best SVM model, left indices, right indices, split class

```

1 function MulticlassSplit( $\mathbf{X}_{\text{scaled}}, \mathcal{Y}, \text{strat}_w, I_{\text{parent}}, \text{measure}_I, \tau_I, n$ ):
2    $k \leftarrow |\text{unique}(\mathcal{Y})|$ 
3    $n_{\text{models}} \leftarrow k$ 
4   for  $j = 1$  to  $n_{\text{models}}$  do
5      $c_j \leftarrow \text{unique}(\mathcal{Y})[j]$  // Target class for OvR
6      $\mathcal{Y}_{\text{binary}} \leftarrow \begin{cases} + & \text{if } y_i = c_j \\ - & \text{otherwise} \end{cases}$ 
7      $w_c \leftarrow \text{calculate\_class\_weights}(\mathcal{Y}_{\text{binary}}, \text{strat}_w)$ 
8      $\text{model}_j \leftarrow \text{fit\_linear\_SVM}(\mathbf{X}_{\text{scaled}}, \mathcal{Y}_{\text{binary}}, w_c)$ 
9     if  $\text{model}_j = \text{null}$  then
10      continue // Skip failed SVM
11      $\mathbf{f}_j \leftarrow \text{model}_j.\text{decision\_values}(\mathbf{X}_{\text{scaled}})$ 
12      $\mathcal{I}_L^j \leftarrow \{i : f_{j,i} > 0\}, \quad \mathcal{I}_R^j \leftarrow \{i : f_{j,i} \leq 0\}$ 
13     if  $|\mathcal{I}_L^j| = 0$  or  $|\mathcal{I}_R^j| = 0$  then
14      continue // Skip degenerate split
15      $I_L^j \leftarrow I(\mathcal{Y}[\mathcal{I}_L^j], \text{measure}_I)$ 
16      $I_R^j \leftarrow I(\mathcal{Y}[\mathcal{I}_R^j], \text{measure}_I)$ 
17      $I_j \leftarrow \frac{|\mathcal{I}_L^j|}{n} \cdot I_L^j + \frac{|\mathcal{I}_R^j|}{n} \cdot I_R^j$ 
18      $IG_j \leftarrow I_{\text{parent}} - I_j$ 
19     if  $IG_j < \tau_I$  then
20      continue // Insufficient information gain
21    $b^* \leftarrow \arg \min_j I_j$  // Select best split
22   if  $\text{model}_{b^*} = \text{null}$  then
23     return (null,  $\emptyset, \emptyset, \text{null}$ )
24   return ( $\text{model}_{b^*}, \mathcal{I}_L^{b^*}, \mathcal{I}_R^{b^*}, c_{b^*}$ )

```

For each candidate split, the classes are divided into two groups, and a binary SVM is trained to separate the corresponding subsets of samples. The quality of the resulting partition is evaluated using an impurity reduction criterion such as information gain. Let $I(\mathcal{Y})$ denote the impurity of the parent node and $I(\mathcal{Y}_L)$ and $I(\mathcal{Y}_R)$ denote the impurities of the resulting child nodes. The information gain is computed as

$$\Delta I = I(\mathcal{Y}) - \left(\frac{|\mathcal{I}_L|}{n} I(\mathcal{Y}_L) + \frac{|\mathcal{I}_R|}{n} I(\mathcal{Y}_R) \right),$$

where n is the number of samples at the node. The candidate split that yields the highest impurity reduction above a predefined threshold τ_I is selected as the optimal split.

If no candidate partition produces a valid improvement or if one of the resulting child nodes is empty, the node is converted into a leaf node.

Oblique Decision Boundary Unlike axis-aligned decision trees, which split data based on a single feature, the SVM-based approach constructs oblique decision boundaries that depend on a linear combination of multiple features. This enables the model to capture more complex decision surfaces while preserving the hierarchical structure and interpretability of the decision tree.

Feature Selection Method At each node of the tree, a subset of candidate features is selected from the full feature set $\mathcal{F}$ to construct the SVM-based oblique split. This process enables the algorithm to focus on informative predictors while maintaining diversity across nodes in the tree. Let $p = |\mathcal{F}|$ denote the total number of available features. At depth d , the algorithm first determines the maximum number of features m_d that may be considered for splitting. This value is computed according to the selected maximum-feature strategy. A subset of up to m_d features is then chosen using a specified feature selection method.

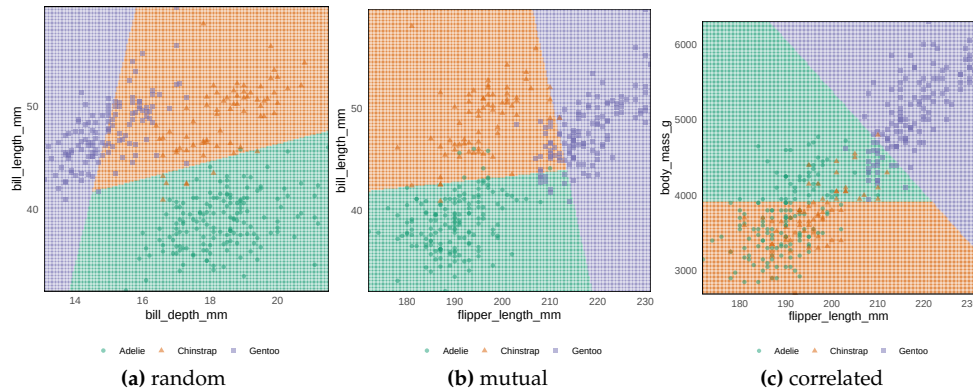


Figure 2: Comparison of random, mutual, and correlated feature selection methods using SVMODT on the Palmer Penguins dataset.

Table 2: Comparison of mean prediction accuracy across feature selection strategies against the Python STree benchmark. All SVMODT models are trained with `max_depth = 15`. *Random*: `feature_method = "random"`, `n_subsets = 10`; *Mutual Information*: `feature_method = "mutual"`; *Correlation*: `feature_method = "cor"`.

Dataset	N	X	L	STree (Benchmark)	Random	Mutual Info	Correlation
WDBC Diagnosis	569	30	2	0.970 ± 0.003	0.967 ± 0.003	0.968 ± 0.006	0.970 ± 0.004
Iris	150	4	3	0.965 ± 0.010	0.955 ± 0.010	0.960 ± 0.009	0.961 ± 0.008
Echocardiogram	131	10	2	0.845 ± 0.008	0.848 ± 0.005	0.851 ± 0.007	0.847 ± 0.007
Fertility	100	9	2	0.874 ± 0.010	0.880 ± 0.003	0.878 ± 0.009	0.878 ± 0.009
Wine	178	13	3	0.959 ± 0.008	0.962 ± 0.016	0.962 ± 0.016	0.961 ± 0.012
Cardiotography-3	2126	21	3	0.903 ± 0.003	0.900 ± 0.007	0.903 ± 0.002	0.899 ± 0.005
Cardiotography-10	2126	21	10	0.795 ± 0.018	0.790 ± 0.008	0.798 ± 0.006	0.793 ± 0.007
Ionosphere	351	33	2	0.892 ± 0.015	0.889 ± 0.020	0.888 ± 0.010	0.881 ± 0.012
Dermatology	366	34	6	0.959 ± 0.004	0.958 ± 0.011	0.969 ± 0.007	0.966 ± 0.003

The algorithm supports multiple feature selection approaches:

- **Random Selection:** A subset of features is sampled uniformly at random from the available feature set. This strategy introduces stochasticity into the tree construction process and helps reduce overfitting.
- **Mutual Information Ranking:** Features are ranked based on their mutual information with the target variable. Mutual information measures the statistical dependency between a feature and the class labels. The top-ranked features are selected for model training. This ranking is implemented using the `FSelectorRcpp` package.
- **Correlation-Based Filtering:** Features are selected based on their correlation with the response variable while reducing redundancy among predictors. This approach aims to retain informative variables while discarding highly correlated or redundant features.

To further encourage diversity across tree nodes, the algorithm incorporates a feature reuse penalty mechanism. Previously selected features are tracked in the set $\mathcal{F}_{\text{used}}$, and a penalty parameter λ_{pen} reduces the probability that these features are repeatedly chosen at deeper levels of the tree. After the candidate feature subset $\mathcal{F}_d$ is selected, the corresponding predictors are standardized at the node level. Features that become constant after preprocessing are removed. If no valid features remain, the node is converted into a leaf node. The selected feature subset is then used to train the SVM model responsible for generating the oblique decision boundary at the node.

For strategies involving random feature subsets, multiple candidate subsets may be evaluated. For each candidate subset, a Support Vector Machine is fitted and the resulting split is assessed using an impurity based criterion such as information gain. The subset that produces the highest impurity reduction is selected for the node. This approach (see Figure 2) enables the tree to identify informative oblique decision boundaries while maintaining computational tractability.

Table 2 compares classification performance of different feature selection strategies evaluated across ten benchmark datasets. Across the datasets, the results indicate that all feature selection strategies achieve competitive performance, with relatively small differences in accuracy. This suggests that the SVM based oblique splitting mechanism remains robust to the specific feature selection approach

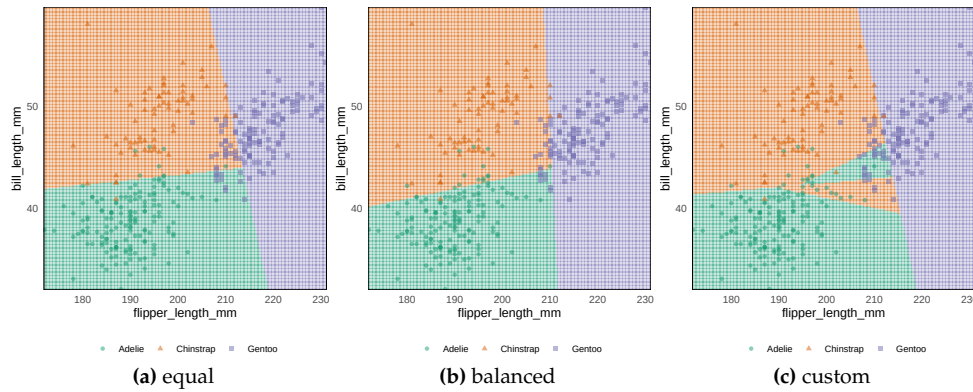


Figure 3: Comparison of equal, balanced, and custom weighting schemes using SVMODT on the Palmer Penguins dataset.

Table 3: Comparison of macro-averaged F1 score (mean $\pm$ sd) across class weighting strategies. All SVMODT models are trained with `max_depth = 15`, `feature_method = "mutual"`, and adaptive `max_features`. *None*: `class_weights = "none"`; *Balanced*: `class_weights = "balanced"`. Bold values denote the highest F1 score for each dataset.

Dataset	N	X	L	Class Imbalance Ratio	STree	None	Balanced
WDBC Diagnosis	569	30	2	0.5938375	0.968 $\pm$ 0.004	0.943 $\pm$ 0.008	0.948 $\pm$ 0.008
Iris	150	4	3	1.0000000	0.965 $\pm$ 0.010	0.963 $\pm$ 0.006	0.945 $\pm$ 0.015
Echocardiogram	131	10	2	0.4886364	0.792 $\pm$ 0.009	0.800 $\pm$ 0.006	0.752 $\pm$ 0.040
Fertility	100	9	2	0.1363636	0.466 $\pm$ 0.003	0.468 $\pm$ 0.001	0.560 $\pm$ 0.041
Wine	178	13	3	0.6760563	0.959 $\pm$ 0.008	0.954 $\pm$ 0.010	0.952 $\pm$ 0.017
Cardiotography-3	2126	21	3	0.1063444	0.812 $\pm$ 0.007	0.766 $\pm$ 0.017	0.827 $\pm$ 0.009
Cardiotography-10	2126	21	10	0.0915371	0.716 $\pm$ 0.011	0.586 $\pm$ 0.071	0.746 $\pm$ 0.011
Ionosphere	351	33	2	0.5600000	0.879 $\pm$ 0.016	0.855 $\pm$ 0.021	0.858 $\pm$ 0.020
Dermatology	366	34	6	0.1785714	0.946 $\pm$ 0.013	0.956 $\pm$ 0.009	0.950 $\pm$ 0.015

used at each node. Random feature selection performs consistently well and achieves accuracy values that are comparable to, and occasionally exceed, the benchmark model. Mutual Information and Correlation based methods provide slightly more stable performance on certain datasets where informative predictors might exhibit stronger statistical relationships with the target variable.

Class Weight Handling To address class imbalance, the algorithm allows four weighting schemes (see Figure 3) :

- **None:** All classes are assigned equal weight, such that $w_c = 1 \forall c \in C$
- **Balanced:** Class weights are inversely proportional to the class frequencies in the training data. The weight assigned to class c is given by $w_c = \frac{n}{K \cdot n_c}$, where n denotes the total number of training samples, K is the number of classes, and n_c is the number of samples belonging to class c .
- **Custom:** Users may specify their own class weights w_c for each class $c \in C$, allowing domain-specific adjustments based on prior knowledge or application requirements.

Table 3 presents the macro-averaged F1 scores obtained under different class weighting strategies across ten benchmark datasets. The results indicate that class weighting has a notable impact on predictive performance across datasets. For datasets exhibiting multiple classes or pronounced class imbalance, such as Cardiotography and Fertility, the choice of weighting strategy substantially affects the macro-averaged F1 score. Specifically, the Balanced weighting scheme frequently improves performance relative to the unweighted (None) configuration. Conversely, in datasets with more balanced class distributions, differences between weighting strategies are smaller, although the highest F1 scores are often achieved with either the Balanced or Custom weights. Overall, the results indicate that selecting an appropriate class weighting strategy is particularly important for datasets with multiple classes or significant class imbalance, while for more balanced datasets, weighting has a minimal impact.

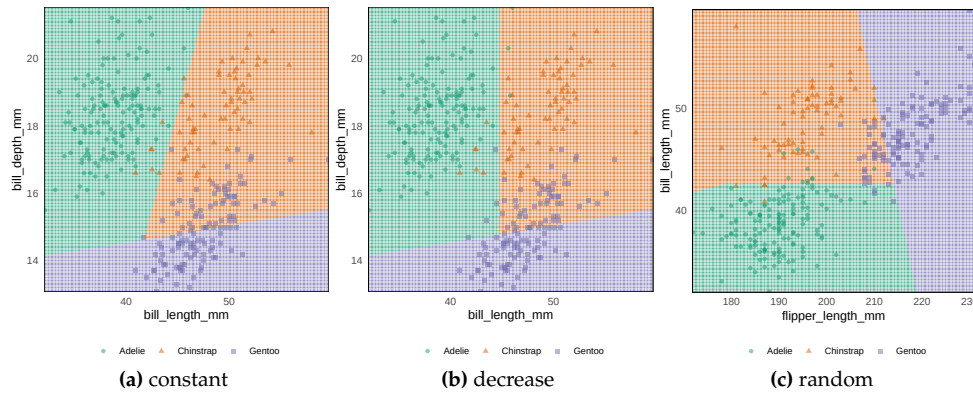


Figure 4: Comparison of constant, decreasing, and random maximum feature selection strategies using SVMODT on the Palmer Penguins dataset.

Table 4: Comparison of mean prediction accuracy (mean $\pm$ sd) across maximum feature selection strategies against the Python STree benchmark. All SVMODT models are trained with `max_depth = 15`, `feature_method = "mutual"`, and adaptive `max_features`. *Constant*: `max_features_strategy = "constant"`; *Decrease*: `max_features_strategy = "decrease"`, `max_features_decrease_rate = 0.5`; *Random*: `max_features_strategy = "random"`, `max_features_random_range = c(0.5, 0.8)`. Bold values denote the highest prediction accuracy per dataset.

Dataset	N	X	L	STree	Constant	Decrease	Random
WDBC Diagnosis	569	30	2	0.970 $\pm$ 0.003	0.969 $\pm$ 0.003	0.970 $\pm$ 0.004	0.967 $\pm$ 0.008
Iris	150	4	3	0.965 $\pm$ 0.010	0.966 $\pm$ 0.006	0.955 $\pm$ 0.003	0.963 $\pm$ 0.006
Echocardiogram	131	10	2	0.845 $\pm$ 0.008	0.846 $\pm$ 0.006	0.846 $\pm$ 0.004	0.850 $\pm$ 0.005
Fertility	100	9	2	0.874 $\pm$ 0.010	0.874 $\pm$ 0.010	0.875 $\pm$ 0.011	0.881 $\pm$ 0.000
Wine	178	13	3	0.959 $\pm$ 0.008	0.978 $\pm$ 0.008	0.976 $\pm$ 0.006	0.959 $\pm$ 0.009
Cardiotography-3	2126	21	3	0.903 $\pm$ 0.003	0.902 $\pm$ 0.004	0.890 $\pm$ 0.002	0.906 $\pm$ 0.005
Cardiotography-10	2126	21	10	0.795 $\pm$ 0.018	0.792 $\pm$ 0.004	0.630 $\pm$ 0.007	0.797 $\pm$ 0.006
Ionosphere	351	33	2	0.892 $\pm$ 0.015	0.896 $\pm$ 0.011	0.892 $\pm$ 0.012	0.884 $\pm$ 0.013
Dermatology	366	34	6	0.959 $\pm$ 0.004	0.972 $\pm$ 0.006	0.979 $\pm$ 0.006	0.971 $\pm$ 0.005

Maximum Feature Selection To control model complexity and encourage feature diversity across tree nodes, the algorithm dynamically limits the number of candidate features considered during split generation (see Figure 4). Let p denote the total number of available predictors. At depth d of the tree, the maximum number of features evaluated is denoted by m_d .

The value of m_d is determined using one of three configurable strategies:

$$m_d = \begin{cases} m_{\text{base}} & \text{constant strategy} \\ \lfloor m_{\text{base}} \cdot \alpha^{d-1} \rfloor & \text{decrease strategy} \\ \text{Uniform}(\lfloor p \cdot \ell_{\min} \rfloor, \lfloor p \cdot \ell_{\max} \rfloor) & \text{random strategy} \end{cases}$$

In the **constant strategy**, the same number of candidate features m_{base} is evaluated at every depth of the tree. This provides consistent computational cost across nodes.

In the **decrease strategy**, the number of candidate features gradually decreases as the tree depth increases. The parameter $\alpha \in (0, 1]$ controls the rate of reduction. This approach allows early splits to explore a larger feature space while later splits focus on a smaller set of predictors.

In the **random strategy**, the number of candidate features is sampled from a uniform distribution bounded by fractions $\ell_{\min}$ and $\ell_{\max}$ of the total feature count p . This introduces additional stochasticity into the tree construction process and may improve generalization by reducing overfitting.

Once m_d is determined, up to m_d features are selected using the specified feature selection method. The resulting subset is then used to train the SVM model responsible for generating the oblique split at the node.

Table 4 summarizes SVMODT performance across different maximum feature selection strategies compared to the Python STree benchmark. Overall, predictive accuracy is largely consistent across strategies, with differences generally within a few percentage points. For most datasets, all three maximum feature strategies—Constant, Decrease, and Random—perform similarly to the STree benchmark, demonstrating the robustness of SVMODT to variations in maximum feature selection.

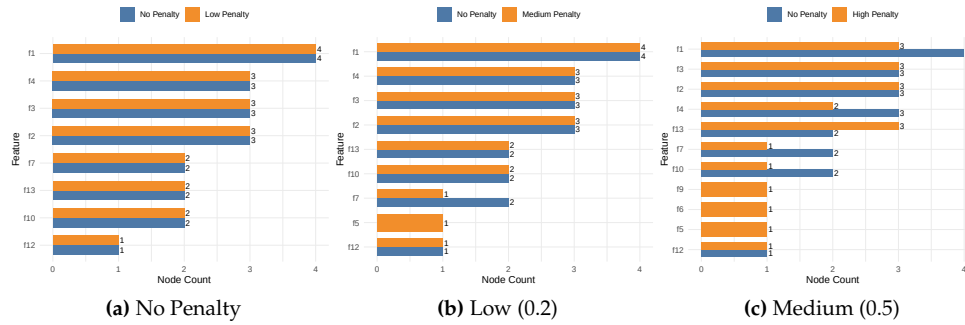


Figure 5: Comparison of Low, Medium and High penalty on repeated features by SVMODT on Wine dataset.

Some datasets, such as Wine, Cardiotography-10, and Dermatology, exhibit notable differences. For example, the Decrease strategy underperforms on Cardiotography-10 relative to Constant and Random, while Random selection yields the highest accuracy for Fertility. In datasets with smaller feature sets or fewer classes, differences between strategies are minimal, indicating that the choice of maximum feature strategy has limited impact when feature dimensionality is low. These results suggest that SVMODT is generally stable across maximum feature selection strategies, but careful selection of the strategy may yield marginal gains on larger or more complex datasets.

Feature Penalty Mechanism To encourage feature diversity across tree nodes and reduce repeated reliance on the same predictors, the algorithm incorporates a feature penalty mechanism during feature selection. This mechanism discourages features that have already been used in earlier splits from being repeatedly selected at deeper levels of the tree (see Figure 5).

Let $\mathcal{F}_{\text{used}}$ denote the set of features that have been previously utilized in the construction of the tree. During the feature selection process at depth d , candidate features are evaluated using the specified feature selection method. A penalty parameter λ_{pen} is applied to features that belong to $\mathcal{F}_{\text{used}}$, reducing their likelihood of being selected.

Formally, if s_j denotes the original relevance score of feature j , the penalized score $\tilde{s}_j$ is defined as

$$\tilde{s}_j = \begin{cases} s_j & \text{if } j \notin \mathcal{F}_{\text{used}} \\ (1 - \lambda_{\text{pen}}) s_j & \text{if } j \in \mathcal{F}_{\text{used}} \end{cases}$$

where $\lambda_{\text{pen}} \in [0, 1]$ controls the strength of the penalty. A value of $\lambda_{\text{pen}} = 0$ corresponds to no penalty, while larger values increasingly discourage the reuse of previously selected features.

After selecting the feature subset $\mathcal{F}_d$ for the current node, the set of used features is updated as

$$\mathcal{F}'_{\text{used}} = \mathcal{F}_{\text{used}} \cup \mathcal{F}_d.$$

This mechanism promotes exploration of different predictors throughout the tree construction process, which can improve model generalization and reduce overfitting caused by repeatedly selecting dominant features.

Table 5 compares SVMODT performance under different feature penalisation strategies compared to the benchmark (None). Overall, the impact of penalising previously used features varies across datasets. On datasets such as WDBC Diagnosis, Wine, and Dermatology, applying low or medium feature penalisation yields slightly higher prediction accuracy than the unpenalised benchmark. For example, Wine achieves the highest accuracy under Medium penalisation (0.951 ± 0.013). Several datasets, including Iris, Echocardiogram, and Fertility, show identical performance across all penalisation levels, suggesting that feature reuse does not influence accuracy in these cases. In datasets with more complex class structures or imbalanced classes, such as Cardiotography-10, high feature penalisation reduces predictive performance (0.739 ± 0.034), indicating that excessive penalisation can hinder model learning. In summary, moderate feature penalisation can provide marginal improvements in predictive accuracy, while excessive penalisation may be detrimental, particularly for datasets with large numbers of classes or complex patterns. Feature penalisation strategies should therefore be tuned according to dataset characteristics to optimise SVMODT performance.

Table 5: Comparison of Mean Prediction Accuracy (mean $\pm$ sd) across feature penalisation strategies. All SVMODT models trained with `max_depth = 15`, `feature_method = "mutual"`, adaptive `max_features`. *None*: `penalize_used_features = FALSE`; *Low*: `penalize_used_features = TRUE`, `feature_penalty_weight = 0.2`; *Medium*: `penalize_used_features = TRUE`, `feature_penalty_weight = 0.5`; *High*: `penalize_used_features = TRUE`, `feature_penalty_weight = 0.8`. Bold denotes the highest accuracy per dataset.

Dataset	N	X	L	None (Benchmark)	Low	Medium	High
WDBC Diagnosis	569	30	2	0.946 $\pm$ 0.005	0.950 $\pm$ 0.004	0.949 $\pm$ 0.007	0.949 $\pm$ 0.007
Iris	150	4	3	0.966 $\pm$ 0.006	0.966 $\pm$ 0.006	0.966 $\pm$ 0.006	0.966 $\pm$ 0.006
Echocardiogram	131	10	2	0.855 $\pm$ 0.001	0.855 $\pm$ 0.001	0.855 $\pm$ 0.001	0.855 $\pm$ 0.001
Fertility	100	9	2	0.881 $\pm$ 0.000	0.881 $\pm$ 0.000	0.881 $\pm$ 0.000	0.881 $\pm$ 0.000
Wine	178	13	3	0.950 $\pm$ 0.015	0.948 $\pm$ 0.016	0.951 $\pm$ 0.013	0.947 $\pm$ 0.011
Cardiotography-3	2126	21	3	0.883 $\pm$ 0.006	0.883 $\pm$ 0.004	0.882 $\pm$ 0.003	0.880 $\pm$ 0.004
Cardiotography-10	2126	21	10	0.782 $\pm$ 0.007	0.775 $\pm$ 0.008	0.776 $\pm$ 0.010	0.739 $\pm$ 0.034
Ionosphere	351	33	2	0.869 $\pm$ 0.018	0.868 $\pm$ 0.016	0.868 $\pm$ 0.017	0.868 $\pm$ 0.017
Dermatology	366	34	6	0.974 $\pm$ 0.006	0.977 $\pm$ 0.006	0.978 $\pm$ 0.006	0.968 $\pm$ 0.009

Performance Evaluation of SVMODT with Hyperparameter Optimisation

Table 6 presents mean prediction accuracy across seven classifiers on ten benchmark datasets. SVMODT* refers to SVMODT trained with dataset-specific hyperparameters identified via a structured multi-seed grid search. Specifically, each candidate hyperparameter configuration was evaluated using three random seeds to estimate both mean accuracy and standard deviation. The configuration with the lowest standard deviation was then selected as the optimal hyperparameter set (see ??) for evaluation across the remaining seeds. STree* denotes the Python STree implementation with dataset-specific kernel and regularisation parameters optimised per dataset. For context, CART baselines were included with shallow (`max_depth = 3`, `cp = 0.01`) and deep (`max_depth = 15`, `cp = 0.001`) configurations.

Overall competitiveness. SVMODT* achieves the highest accuracy on three datasets (Iris, Dermatology, and Statlog Australian Credit), matching or exceeding all competitors including STree* on these benchmarks. Notably, SVMODT* relies exclusively on a linear kernel at each node, whereas STree* employs non-linear kernels on several datasets—specifically RBF on Ionosphere and polynomial on Echocardiogram—providing STree* with a representational advantage in those cases.

Performance relative to STree*. Despite this kernel asymmetry, SVMODT* remains competitive across most datasets. STree* outperforms SVMODT* on WDBC, Fertility, and Ionosphere, where non-linear kernels or heavily regularised configurations (`C = 0.05–7`, RBF/poly kernels) confer clear advantages. On datasets where STree* uses a linear kernel (Iris, Wine, CTG-3, CTG-10, Dermatology), SVMODT* either matches or surpasses it, suggesting that the observed performance gap is largely attributable to kernel expressiveness rather than the oblique tree construction strategy.

Performance relative to default SVMODT. Hyperparameter tuning via grid search provides measurable improvements on Iris (+0.005), Dermatology (+0.007), and CTG-3 (+0.005). However, SVMODT* underperforms the default configuration on WDBC (-0.004), Echocardiogram (-0.011), and Fertility (-0.046). The pronounced reduction on Fertility is notable: the grid search-selected configuration (correlation-based feature selection, balanced class weights, random maximum feature strategy, `penalty_weight = 0.6`) generalises poorly compared to the default `max_depth = 15` with no additional constraints, indicating potential overfitting on small datasets (`n = 100`). This underscores the robustness of the default SVMODT configuration as a strong baseline.

Performance relative to tree-based baselines. Both SVMODT variants substantially outperform CART (shallow and deep) on most datasets. The exception is CTG-10, where deep CART (0.821) surpasses all oblique methods, including SVMODT* (0.797) and STree* (0.806). This aligns with the known behaviour of axis-aligned trees on datasets with many classes: greedy feature splitting in deep trees can capture high-class-count structure effectively. On Dermatology (six classes), SVMODT* achieves the best result overall (0.979), demonstrating strength in multiclass, high-dimensional feature settings.

Hyperparameter patterns in SVMODT*. Examination of the optimal configurations reveals a preference for shallow trees (`max_depth = 5–10`), with random or decreasing maximum feature strategies selected in most datasets. Feature penalisation is active in six of nine datasets, highlighting the benefit of feature diversity in oblique trees. Correlation-based feature selection dominates across five datasets, followed by random (three datasets) and mutual information (one dataset). The `min_impurity_decrease` parameter is set to 0.01 or higher for CTG-10 and Ionosphere, acting as a regulariser to prevent spurious splits in higher-dimensional or noisier settings.

SVMODT* provides competitive predictive performance across benchmark datasets, achieving top accuracy in several cases despite using linear node kernels. Hyperparameter tuning can yield modest gains, but default configurations already provide a strong baseline, particularly on small or low-dimensional datasets. Oblique trees with feature diversity and controlled regularisation show particular advantages for multiclass and high-dimensional problems.

Table 6: Comparison of Mean Prediction Accuracy (mean $\pm$ sd) across classifiers. *STree** denotes Python STree with dataset-specific hyperparameters. *SVMODT** denotes SVMODT with tuned hyperparameters from grid search. Bold denotes the highest accuracy per dataset.

Dataset	N	X	L	CART (shallow)	CART (deep)	SVM (linear)	Log. Reg.	STree*	SVMODT (Default)	SVMODT*
WDBC Diagnosis	569	30	2	0.929 $\pm$ 0.007	0.931 $\pm$ 0.008	0.970 $\pm$ 0.004	0.946 $\pm$ 0.007	0.973 $\pm$ 0.004	0.969 $\pm$ 0.003	0.965 $\pm$ 0.005
Iris	150	4	3	0.929 $\pm$ 0.008	0.929 $\pm$ 0.008	0.965 $\pm$ 0.010	0.962 $\pm$ 0.011	0.965 $\pm$ 0.010	0.966 $\pm$ 0.006	0.971 $\pm$ 0.006
Echocardiogram	131	10	2	0.817 $\pm$ 0.015	0.802 $\pm$ 0.012	0.846 $\pm$ 0.004	0.791 $\pm$ 0.018	0.844 $\pm$ 0.011	0.846 $\pm$ 0.006	0.835 $\pm$ 0.015
Fertility	100	9	2	0.833 $\pm$ 0.026	0.833 $\pm$ 0.026	0.874 $\pm$ 0.010	0.840 $\pm$ 0.014	0.881 $\pm$ 0.000	0.874 $\pm$ 0.010	0.828 $\pm$ 0.024
Wine	178	12	3	0.884 $\pm$ 0.020	0.884 $\pm$ 0.020	0.961 $\pm$ 0.005	0.982 $\pm$ 0.006	0.964 $\pm$ 0.010	0.978 $\pm$ 0.008	0.970 $\pm$ 0.008
Cardiotography-3	2126	21	3	0.901 $\pm$ 0.003	0.918 $\pm$ 0.005	0.897 $\pm$ 0.002	0.892 $\pm$ 0.003	0.903 $\pm$ 0.003	0.902 $\pm$ 0.004	0.907 $\pm$ 0.004
Cardiotography-10	2126	21	10	0.717 $\pm$ 0.003	0.821 $\pm$ 0.003	0.820 $\pm$ 0.007	0.820 $\pm$ 0.003	0.806 $\pm$ 0.007	0.792 $\pm$ 0.004	0.797 $\pm$ 0.006
Ionosphere	351	33	2	0.892 $\pm$ 0.005	0.868 $\pm$ 0.011	0.878 $\pm$ 0.008	0.869 $\pm$ 0.010	0.949 $\pm$ 0.006	0.896 $\pm$ 0.011	0.886 $\pm$ 0.012
Dermatology	366	34	6	0.777 $\pm$ 0.004	0.928 $\pm$ 0.006	0.966 $\pm$ 0.005	0.965 $\pm$ 0.006	0.966 $\pm$ 0.005	0.972 $\pm$ 0.006	0.979 $\pm$ 0.004

4 Acknowledgements

The authors would like to thank Professor **Natalia Da Silva** and Professor **Dianne Helen Cook** for their constructive discussions and valuable insights regarding oblique decision trees and their implementations in R. The authors also acknowledges the use of OpenAI's GPT-5 model (ChatGPT) for editorial assistance and technical writing support during the preparation of this manuscript. All conceptual development, analysis, data processing, coding, and interpretation of results were performed independently by the author. The AI tool was used solely to refine language clarity, improve structure, and ensure stylistic consistency in the documentation.

5 Appendix

Algorithm 5: STree (Multi-class SVM-based oblique decision tree)

```

Input:  $\mathcal{D}' = \{(\vec{x}_i, y_i)\}_{i=1}^n$ : Data
Output: tree: the root node of an SVM-based oblique decision tree

1 function STree( $\mathcal{D}'$ ):
2   if stopping_condition( $\{y_i\}_{i=1}^t$ ) then
3     return create_leaf_node(mode( $\{y_i\}_{i=1}^t$ ))           // Leaf node
4    $k' \leftarrow \text{num\_different\_labels}(\{y_i\}_{i=1}^t)$ 
5      $r \leftarrow \frac{k'(k'-1)}{2}$ 
6      $\mathcal{Y}' \leftarrow \{y'_1, \dots, y'_{k'}\}$                        // set of labels
7    $I_{all} \leftarrow I(\{y_i\}_{i=1}^t)$                        //  $I(\cdot)$  is an information theory measure
8   if (OvO) then
9      $n_{models} \leftarrow r$ 
10  else
11     $n_{models} \leftarrow k'$                                // OvR
12  for  $j = 1$  to  $n_{models}$  do
13    if ( $j = k' = 2$ ) then
14      break                                             // Two labels, one SVM is enough
15    if (OvO) then
16      Let  $c_a$  and  $c_b$  the pair of labels corresponding to index  $j$ 
17       $models[j] \leftarrow \text{SVM}(\mathcal{D}'^{\downarrow c_a} \cup \mathcal{D}'^{\downarrow c_b})$ 
18    else
19       $models[j] \leftarrow \text{SVM}(\mathcal{D}')$  using binary class  $y'_j$  (+) vs rest (-) // OvR
20   $\mathcal{D}'^+, \mathcal{D}'^- \leftarrow models[j](\vec{x}_j) \quad \forall (\vec{x}) \in \mathcal{D}'$ 
21   $I_j \leftarrow \frac{|\mathcal{D}'^+|}{|\mathcal{D}'|} I(\{y_i\}_{i=1}^t | \mathcal{D}'^+) + \frac{|\mathcal{D}'^-|}{|\mathcal{D}'|} I(\{y_i\}_{i=1}^t | \mathcal{D}'^-)$ 
22   $IG_j \leftarrow I_{all} - I_j$ 
23   $b^* = \arg \max_{j=1, \dots, n_{models}} IG_j$ 
24  if  $IG_{b^*} > 0$  then
25     $node \leftarrow \text{create\_node}(models[b^*])$ 
26     $node.left \leftarrow \text{STree}(\mathcal{D}'^+)$ 
27     $node.right \leftarrow \text{STree}(\mathcal{D}'^-)$ 
28  else
29    return create_leaf_node(mode( $\{y_i\}_{i=1}^t$ ))       // No split gain
30  return node

```

dataset	max_depth	feature_method	class_weights	max_features_strategy	penalize_used_features	feature_penalty_weight	min_impurity_decrease	max_features
wdbc	10	random	none	random	TRUE	0.6	0.00	NA
iris	5	cor	none	decrease	FALSE	0.3	0.00	2
echocardiogram	5	random	balanced	decrease	TRUE	0.6	0.01	3
fertility	5	cor	balanced	random	TRUE	0.6	0.00	NA
wine	5	cor	none	constant	FALSE	0.3	0.00	13
ctg3	15	cor	balanced	random	TRUE	0.3	0.01	NA
ctg10	10	mutual	balanced	random	TRUE	0.6	0.05	NA
ionosphere	5	random	none	random	FALSE	0.3	0.01	NA
dermatology	10	cor	none	decrease	FALSE	0.3	0.00	34

Bibliography

- M. Bala and R. Agrawal. Optimal decision tree based multi-class support vector machine. *Informatica*, 35(2), 2011. [p1, 4]
- K. Bennett and J. Blue. A support vector machine approach to decision trees. In *1998 IEEE International Joint Conference on Neural Networks Proceedings. IEEE World Congress on Computational Intelligence (Cat. No.98CH36227)*, volume 3, pages 2396–2401 vol.3, 1998. doi: 10.1109/IJCNN.1998.687237. [p1, 4]
- L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone. *Classification and Regression Trees*. Wadsworth, 1984. URL <http://lyle.smu.edu/~mhd/8331f06/cart.pdf>. [p1]
- C. E. Brodley and P. E. Utgoff. Multivariate decision trees. *Machine Learning*, 19:45–77, 1995. URL <https://api.semanticscholar.org/CorpusID:16836572>. [p2, 3]
- S. Bulò and P. Kotschieder. Neural decision forests for semantic image labelling. In *2014 IEEE Conference on Computer Vision and Pattern Recognition*, pages 81–88, 2014. doi: 10.1109/CVPR.2014.18. [p3]
- L. Cañete, R. Monroy, and M. Medina-Pérez. A review and experimental comparison of multivariate decision trees. *IEEE Access*, PP:1–1, 08 2021. doi: 10.1109/ACCESS.2021.3102239. [p1]
- J. Cervantes, F. Garcia-Lamont, L. Rodríguez-Mazahua, and A. Lopez. A comprehensive survey on support vector machine classification: Applications, challenges and trends. *Neurocomputing*, 408:189–215, 2020. ISSN 0925-2312. doi: <https://doi.org/10.1016/j.neucom.2019.10.118>. URL <https://www.sciencedirect.com/science/article/pii/S0925231220307153>. [p4]
- M. Chabbouh, S. Bechikh, E. Mezura-Montes, and L. Ben Said. Evolutionary optimization of the area under precision-recall curve for classifying imbalanced multi-class data. *Journal of Heuristics*, 31, 01 2025. doi: 10.1007/s10732-024-09544-z. [p3]
- C. Cortes and V. Vapnik. Support-vector networks. *Machine learning*, 20(3):273–297, 1995. [p1, 3]
- N. Cristianini. An introduction to support vector machines and other kernel-based learning methods, 2000. [p1, 3]
- J.-x. Dong, A. Krzyzak, and C. Y. Suen. Fast svm training algorithm with decomposition on very large data sets. *IEEE transactions on pattern analysis and machine intelligence*, 27(4):603–618, 2005. [p4]
- M. R. Frean. *Small nets and short paths: Optimising neural computation*. PhD thesis, 1990. [p3]
- M. Friedl and C. Brodley. Decision tree classification of land cover from remotely sensed data. *Remote Sensing of Environment*, 61(3):399–409, 1997. ISSN 0034-4257. doi: [https://doi.org/10.1016/S0034-4257\(97\)00049-7](https://doi.org/10.1016/S0034-4257(97)00049-7). URL <https://www.sciencedirect.com/science/article/pii/S0034425797000497>. [p3]
- J. H. Friedman and J. W. Tukey. A projection pursuit algorithm for exploratory data analysis. *IEEE Transactions on computers*, 100(9):881–890, 2006. [p3]
- S. I. Gallant. Optimal linear discriminants. *Eighth International Conference on Pattern Recognition*, pages 849–852, 1986. URL <https://cir.nii.ac.jp/crid/1573668924476144256>. [p3]
- M. Ganaie, M. Tanveer, P. Suganthan, and V. Snasel. Oblique and rotation double random forest. *Neural Networks*, 153:496–517, 2022. ISSN 0893-6080. doi: <https://doi.org/10.1016/j.neunet.2022.06.012>. URL <https://www.sciencedirect.com/science/article/pii/S0893608022002258>. [p1]
- C.-W. Hsu and C.-J. Lin. A comparison of methods for multiclass support vector machines. *IEEE transactions on Neural Networks*, 13(2):415–425, 2002. [p4]
- J. Kittler. Feature selection and extraction. *Handbook of Pattern Recognition and Image Processing*, pages 59–83, 1986. URL <https://cir.nii.ac.jp/crid/1573950400671608448>. [p3]
- M. Koziol and M. Wozniak. *Multivariate Decision Trees vs. Univariate Ones*, pages 275–284. Springer Berlin Heidelberg, Berlin, Heidelberg, 2009. ISBN 978-3-540-93905-4. doi: 10.1007/978-3-540-93905-4_33. URL https://doi.org/10.1007/978-3-540-93905-4_33. [p3]
- J. B. Kruskal. Toward a practical method which helps uncover the structure of a set of multivariate observations by finding the linear transformation which optimizes a new “index of condensation”. In *Statistical computation*, pages 427–440. Elsevier, 1969. [p3]

- T. D. Lemmond, B. Y. Chen, A. O. Hatch, and W. G. Hanley. *An Extended Study of the Discriminant Random Forest*, pages 123–146. Springer US, Boston, MA, 2010. ISBN 978-1-4419-1280-0. doi: 10.1007/978-1-4419-1280-0_6. URL https://doi.org/10.1007/978-1-4419-1280-0_6. [p3]
- H. Liu and R. Setiono. Feature transformation and multivariate decision tree induction. In S. Arikawa and H. Motoda, editors, *Discovery Science*, pages 279–291, Berlin, Heidelberg, 1998. Springer Berlin Heidelberg. ISBN 978-3-540-49292-4. [p3]
- V. Menkovski, I. T. Christou, and S. Efremidis. Oblique decision trees using embedded support vector machines in classifier ensembles. In *2008 7th IEEE International Conference on Cybernetic Intelligent Systems*, pages 1–6, 2008. doi: 10.1109/UKRICIS.2008.4798937. [p1]
- B. H. Menze, B. M. Kelm, D. N. Splitthoff, U. Koethe, and F. A. Hamprecht. On oblique random forests. In D. Gunopulos, T. Hofmann, D. Malerba, and M. Vazirgiannis, editors, *Machine Learning and Knowledge Discovery in Databases*, pages 453–469, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg. ISBN 978-3-642-23783-6. [p3]
- J. Mingers. An empirical comparison of pruning methods for decision tree induction. *Machine Learning*, 4:227–243, 01 1989. doi: 10.1023/A:1022604100933. [p3]
- R. Montañana, J. A. Gámez, and J. M. Puerta. Stree: A single multi-class oblique decision tree based on support vector machines. In E. Alba, G. Luque, F. Chicano, C. Cotta, D. Camacho, M. Ojeda-Aciego, S. Montes, A. Troncoso, J. Riquelme, and R. Gil-Merino, editors, *Advances in Artificial Intelligence*, pages 54–64, Cham, 2021. Springer International Publishing. ISBN 978-3-030-85713-4. [p4]
- R. Montañana, J. A. Gámez, and J. M. Puerta. Stree: a single multi-class oblique decision tree based on support vector machines, 2021. [p1]
- M. Nanda, K. Seminar, D. Nandika, and A. Maddu. A comparison study of kernel functions in the support vector machine and a comparison study of kernel functions in the support vector machine and its application for termite detection, 2018. [p4]
- G. Pagallo and D. Haussler. Boolean feature discovery in empirical learning. *Machine Learning*, 5:71–99, 1990. URL <https://api.semanticscholar.org/CorpusID:5661437>. [p1, 3]
- F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. [p1]
- J. R. Quinlan. Induction of decision trees. *Machine Learning*, 1:81–106, 1986. URL <https://api.semanticscholar.org/CorpusID:189902138>. [p2]
- J. R. Quinlan. *C4. 5: programs for machine learning*. Elsevier, 2014. [p1, 3]
- C. Schaffer. Deconstructing the digit recognition problem. In D. Sleeman and P. Edwards, editors, *Machine Learning Proceedings 1992*, pages 394–399. Morgan Kaufmann, San Francisco (CA), 1992. ISBN 978-1-55860-247-2. doi: <https://doi.org/10.1016/B978-1-55860-247-2.50056-5>. URL <https://www.sciencedirect.com/science/article/pii/B9781558602472500565>. [p3]
- F. Takahashi and S. Abe. Decision-tree-based multiclass support vector machines. volume 3, pages 1418 – 1422 vol.3, 12 2002. ISBN 981-04-7524-1. doi: 10.1109/ICONIP.2002.1202854. [p1, 4]
- A. K. Y. Truong. Fast growing and interpretable oblique trees via logistic regression models. 2009. URL <https://api.semanticscholar.org/CorpusID:57884720>. [p3]
- P. C. Young. Recursive estimation and time-series analysis: An introduction. 1984. URL <https://api.semanticscholar.org/CorpusID:60181335>. [p3]
- L. Zhang and P. N. Suganthan. Oblique decision tree ensemble via multisurface proximal support vector machine. *IEEE Transactions on Cybernetics*, 45(10):2165–2176, 2015. doi: 10.1109/TCYB.2014.2366468. [p3]

Aneesh Agarwal
Monash University

aaga0022@student.monash.edu

Jack Jewson
Monash University
Department of Econometrics and Business Statistics, Monash University, Australia
Jack.Jewson@monash.edu

Erik Sverdrup
Monash University
Department of Econometrics and Business Statistics, Monash University, Australia
Erik.Sverdrup@monash.edu